AI Manufacturing Copilot

Project Report - Progress, Verification & Improvement Roadmap

25ARE471 Introduction to Generative AI | End-Semester Project | Generated 23 October 2026

A production-oriented GenAI copilot for manufacturing **process planning** and **visual quality inspection**. It grounds plans and answers in plant knowledge (RAG), validates every draft against deterministic YAML rule packs, and gates all critical steps behind human approval, with full traceability and feedback capture.

1. Executive Summary

Component	Status of work delivered
Backend	FastAPI + SQLAlchemy, ~30 REST endpoints; JWT auth with admin roles, rate limiting, Prometheus metrics, structured error handling
GenAI core	RAG pipeline (chunker, embeddings, searchable/Chroma vector store); LLM + VLM gateway with Ollama structured outputs; JSON self-repair loop
Deterministic guardrails	YAML rule packs (shop_rules, glossary) validated by a rule engine; every operation must carry a knowledge citation
Inspection	OpenCV image quality gate + VLM defect analysis with compliant schema, confidence thresholds and audit trail
Frontend	React 18 + TypeScript (Vite): Dashboard, Plans, PlanDetail, Inspections, Ask, Documents, Auth pages; typecheck + prod build clean
Verification	Backend test suite: **31/31 passing** (re-verified today, pytest exit 0); end-to-end script scripts/e2e_verify.sh green (5/5)
Local model stack	Portable Ollama with qwen2.5vl:3b (chat + vision) and nomic-embed-text - fully offline, zero cloud cost

The project runs fully offline today: register the first account as plant admin, seed the 6-document demo knowledge base, and every flow (planning, inspection, grounded/general Q&A, feedback, dashboard) demonstrably works end-to-end.

2. Work Completed

2.1 Backend API and platform

Routing domain modules (auth, documents, plans, inspections, plus feedback/Q&A/dashboard/admin in misc): ~30 REST endpoints covering the full lifecycle.

Security: JWT authentication (PyJWT + bcrypt), first-user-becomes-admin bootstrap, in-memory token-bucket rate limiting, request size limits.

Observability: prometheus-client /metrics, JSON logging, uniform AppError-based error responses.

Data layer: SQLAlchemy 2 models for users, documents, plans, inspections, feedback and audit records; SQLite for local dev, PostgreSQL targeted for prod.

2.2 Retrieval-Augmented Generation (RAG)

Ingestion accepts pasted text and .txt/.md/.**pdf**/.**docx** uploads (pypdf, python-docx); chunking into overlap windows.

Pluggable embeddings: OpenAI-compatible APIs for real models with automatic fallback to offline hashing embeddings at boot (never crashes if the provider is unreachable).

Searchable-similarity vector store with a Chroma option for larger corpora; startup auto re-embeds ingested documents when the embedding model changes.

Grounded answers cite retrieved chunks as [K#] with confidence scores.

2.3 Planner, rules engine and LLM gateway

LLM gateway converts requirements into a **structured process-plan JSON** (operations, machines, tooling, inspection checkpoints, references).

Ollama **structured outputs** (response_format: json_schema) enforce schema-valid JSON at decode time, with graceful fallback on endpoints that reject the feature.

Bounded **self-repair loop** feeds pydantic validation errors back to the model; lenient coercion puts malformed fields (e.g. "High" severity, dict references) back in shape instead of failing.

Deterministic **rule packs** (data/rulepacks/shop_rules.yaml, glossary.yaml) load machine capability and forbidden-process constraints without code changes; Horizontal Bandsaw added; detector ignores "grinding stock" allowance phrasing.

Citation-attachment fallback: keyword overlap between operations and retrieved chunks guarantees every operation carries a source even when the small model omits [K#] markers.

2.4 Quality inspection and Q&A;

OpenCV-based image **quality gate** (blur/exposure checks) before VLM analysis; verdicts captured with confidence, evidence and audit trail.

Q&A runs **two modes**: grounded (answers strictly from retrieved document chunks, with citations) and general (conversationally with thread history when the off-topic baseline gate finds no KB match; the UI labels it "not from your documents").

A hard refusal fires only when no real LLM is available or the call fails - honesty over guessing.

2.5 Frontend, docs and deployment scaffolding

React + TypeScript SPA: Dashboard, Plans, PlanDetail, Inspections, Ask (chat-style Q&A), Documents, Auth; clean typecheck and production build.

Docs: README with full setup/architecture, docs/HANDOFF.md status handoff, concepts guide PDF (LaTeX source), and scripted demo (scripts/demo.py).

Deployment: Dockerfiles for backend/frontend plus dev and prod docker-compose files (nginx fronting the API); a Makefile for common tasks.

3. Verification Status

Check	Detail	Status
Backend unit/API tests	31/31 passing - pytest exit 0, re-run during this report	Verified
Frontend typecheck + build	npm run build clean	Reported in handoff
E2E walking skeleton	scripts/e2e_verify.sh: login, plan creation + validation, grounded Q&A, off-topic general chat, VLM inspection - 5/5 green	Reported
Knowledge base	6 demo documents seeded (2 SOPs, routing, standard, lesson, control plan)	Verified
Local model serving	Ollama v0.34.4 on port 11434; qwen2.5vl:3b + nomic-embed-text pulled (ollama_pull.log shows the 3.2 GB pull)	Verified
Docker prod deployment	Compose files authored - NOT executed (no Docker exercised in this workspace)	Pending

Check	Detail	Status
Evaluation against project KPIs	Not yet performed against the presentation's metrics plan (Sec. 4)	Pending

4. Known Limitations

Model quality depends on the local qwen2.5vl:3b model; an OpenAI-compatible cloud key had zero credits, so thresholds (inspection, confidence, off-topic gate) still need calibration on real plant images and documents.

Rate limiter is in-memory only - it resets on restart and is not sync across replicas.

The vector index rebuilds at every startup - acceptable to roughly 50k chunks, beyond that the Chroma backend should be enabled.

Production Docker stack is authored but never exercised end-to-end; no TLS, no backup policy, no secrets manager yet.

Workspace hygiene: project is not under version control (no git repo/commit history, no CI), and log/output artifacts (backend.log, ollama logs) sit at the repo root.

Evaluation defined in the project deck (Recall@k, citation correctness, precision/recall/IoU,unsupported-claim rate, expert 1-5 rubric) is not yet measured.

One small defect noticed during checks: the backup copy of the test database (backend/data/test_copilot.db) and generated log files are present in the workspace and should be gitignored, not retained - see R1.1.

5. Improvements - Prioritised Roadmap

5.1 Priority 1 - Quality, safety and reproducibility

Git + CI: init the repository, gitignore logs/test DB/.venv, add GitHub Actions running ruff, mypy, pytest and the frontend build on every push (all tooling is already in pyproject dev deps).

Quantitative evaluation harness: build a small golden dataset (labeled defect images + expert-reviewed plans) and a pytest-optional eval script reporting Recall@k, citation correctness, unsupported-claim rate, precision/recall, plus the 1-5 expert rubric from the deck.

Threshold calibration: tune inspection confidence, quality-gate and off-topic baseline gates on real images; log distributions in Prometheus for continuous tuning.

Strict typing sweep: raise mypy/ruff coverage in app/services (the hot JSON-coercion paths deserve unit tests for malformed-model outputs).

5.2 Priority 2 - Production hardening

Deploy & validate the prod compose stack once on a real Docker host: set SECRET_KEY/POSTGRES_PASSWORD, add TLS (nginx or cloud LB), run /health checks, and execute e2e_verify.sh against the deployed origin.

Redis-backed rate limiting and shared session state before any multi-replica deploy.

Persistent vector store: prefer Chroma (or pgvector) to avoid full re-embed at boot; schedule incremental re-indexing.

Alembic migrations in the deploy path (dependency present, migrations unwired), plus automated Postgres/upload backups with documented retention.

Upload storage target: move document/image uploads to an object store or mounted volume; audit retention configuration (regulatory requirement).

5.3 Priority 3 - Product depth and model options

Real plant data: replace the demo corpus with the client's SOPs, routing sheets, control plans and lessons learned (designation list in HANDOFF.md Sec. 2); update machine master data in shop_rules.yaml accordingly.

Plan versioning with diffs: keep version history per plan with visible engineer-edit diffs feeding the structured feedback loop.

Stronger vision: defect localization with IoU reporting (currently verdict-level only), augmentation Training for poor image quality.

Manufacturing memory: a curated course of approved plans + engineer corrections that planners can retrieve (deck innovation #1) - today the feedback is captured, but not reused in retrieval.

SSO (OIDC) for plant-wide adoption and finer-grained roles (planner vs inspector vs viewer).

6. Immediate Next Steps (this week)

1. Put the project under git with a CI pipeline (R1.1).
2. Run docker compose -f docker-compose.prod.yml once and fix whatever breaks; capture findings in docs/HANDOFF.md (R2.1).
3. Collect 10-20 labeled inspection images and 2-3 expert-reviewed plans to bootstrap the evaluation harness (R1.2).
4. Decide the LLM path: cloud key with credits vs. larger local model (e.g. qwen2.5vl:7b on more VRAM) - this gates real-world answer quality.

All claims above are grounded in the actual project artifacts: backend/app (28 Python modules incl. 6-dimension service split), frontend/src pages and styles, backend/tests (7 files, 31 tests), data rule packs and seed corpus, docs/HANDOFF.md, Dockerfiles/compose files, and run logs. Backend test status (31/31) was re-verified by executing pytest during report generation.